



Análise e Projeto Orientado a Objetos

uma abordagem prática

Prof. Me. Ricardo Francisco de Pierre Satin, ITIL, CSM, PMP

Apresentação

- Graduado em Computação – UEM – 2002
- Pós-graduado em desenvolvimento Java – Cesumar – 2004
- Pós-graduado em GP – FCV – 2006
- Mestre em Engenharia de Software – UTFPR – 2015
- Certificações:
 - ITIL – 2007
 - PMP – 2008
 - CSM – 2013
- Consultor em gerenciamento de projetos, processos e engenharia de software;
- Professor Universitário (graduação e pós);
- Escritor, e;
- Empreendedor (I.A., RA, RV, B.I. e Mobile).

Introdução

- Análise e Projeto OO
 - Uma introdução geral;
 - A análise;
 - O projeto;
 - Tópicos avançados
 - Cenários de utilização.

Introdução da unidade I

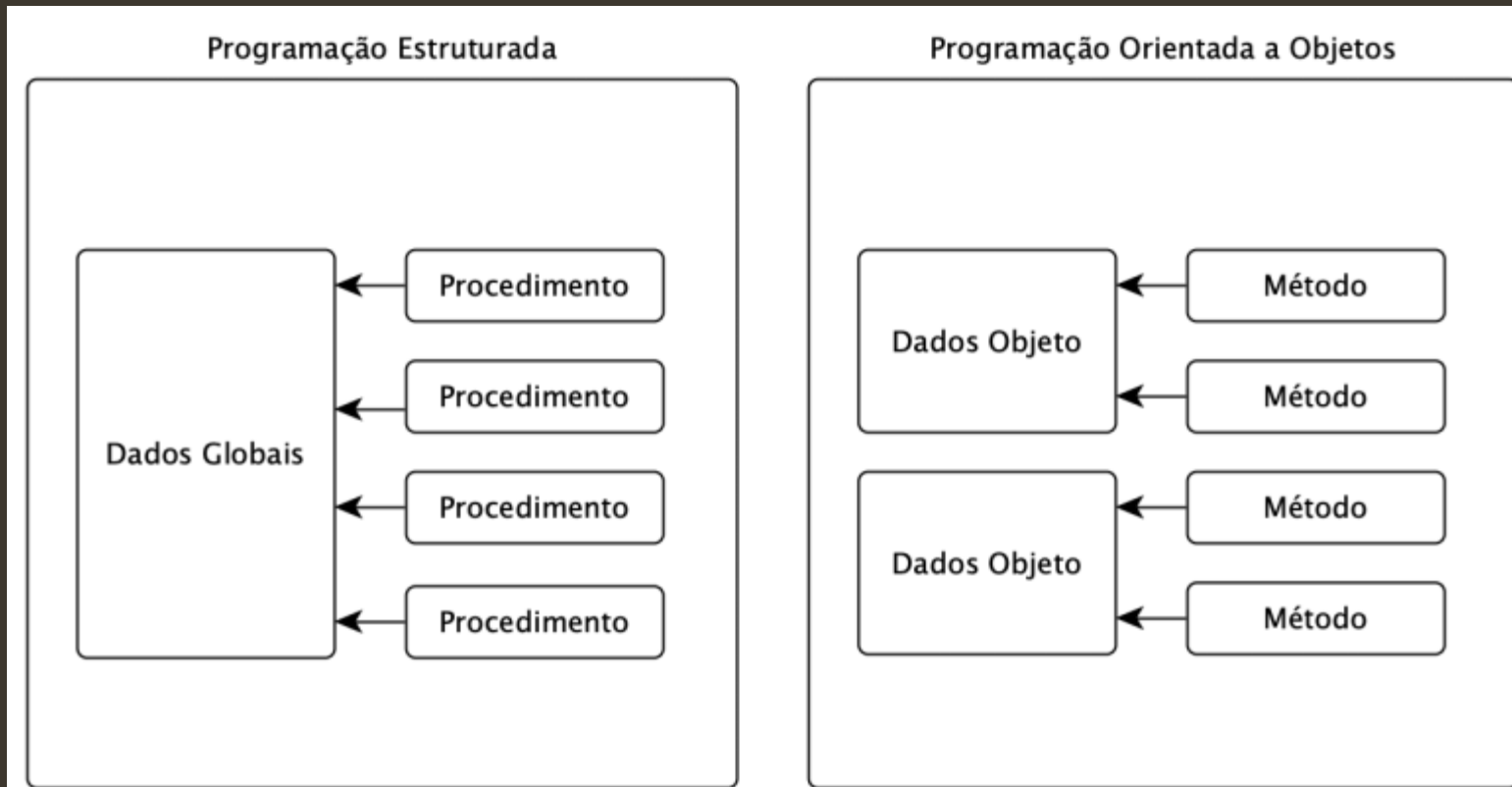
- Como objetivo geral esta unidade iremos:
 - Aprender os conceitos fundamentais de orientação a objeto
 - Aprender os conceitos fundamentais de análise e projetos orientado a objeto
 - Aprender os modelos iterativo, incremental e ágil

Introdução tópico 1 – Conceitos Fundamentais de Orientação a Objetos

- Vamos entender os conceitos principais do que vem a ser Orientação a Objeto (OO).

Conceitos fundamentais de orientação a objeto

- O desenvolvimento de SW é muito amplo (<> linguagens, <> paradigmas);



Conceitos fundamentais de orientação a objeto

- Estruturada
 - Sequencial, sem desvios, maior liberdade com o hardware(desempenho);
- OO
 - Pontos mais interessantes no contexto de aplicações modernas;
 - Reuso de software;
 - Melhor representação do mundo real

O que é necessário para dizer que uma linguagem implementa o paradigma OO?

Conceitos fundamentais de orientação a objeto

- Abstração
 - Aqui um dos conceitos mais importantes dentro de qualquer linguagem OO;
 - Trata da abstração de como um objeto do mundo real vai ser representado no sistema;
 - Três pontos que devem ser levados em conta para abstração
 - Identidade única (pacotes, ou *namespace*);
 - Características dos objetos (propriedades);
 - Quais as ações esse objeto irá executar.

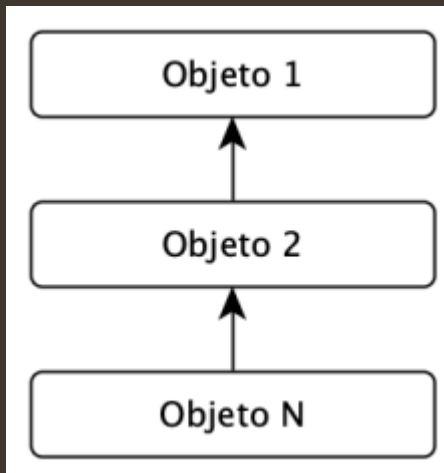
Essas ações ou eventos são chamados métodos

Conceitos fundamentais de orientação a objeto

- Encapsulamento
 - Trás uma característica de segurança a aplicação;
 - Pois esconde as propriedades criando uma espécie de caixa preta
 - A maioria das linguagens implementa o encapsulamento por meio dos métodos getters e setters, impedindo assim o acesso direto às propriedades criando uma camada de segurança.
 - Há o encapsulamento quanto as funcionalidade que estão contidas dentro do seus métodos.
(ligar a TV, microondas).

Conceitos fundamentais de orientação a objeto

- Herança
 - Uma das grandes vantagens da OO é o reuso de código e muito disso se dá por causa da herança
 - Exemplo disso vemos em nossas famílias (filhos, pais, avós...)
 - Herança múltipla C++, C# (Object), Interfaces;



Conceitos fundamentais de orientação a objeto

- Polimorfismo
 - Na natureza, vemos animais que são capazes de alterar sua forma conforme a necessidade, e é dessa ideia que vem o polimorfismo na orientação a objetos
 - Os objetos filhos herdam as características e ações de seus ancestrais.
 - Entretanto, é importante observar que o polimorfismo consiste na alteração do funcionamento interno de um método herdado de um objeto pai, pois pode haver situação que exija um comportamento diferente.
 - Exemplo: Apesar de uma TV, Geladeira e Micro-ondas serem “Eletrodomésticos” o comportamento executado do método “Ligar()” em cada caso é diferente, exigindo que seja feita uma reescrita desse método.
 - As linguagens acabam implementando uma forma de uso desse recurso de forma diferente, por exemplo, em C# você precisa identificar esse método na classe ancestral com o uso da palavra reservada virtual, marcando esse método como virtual e na classe filha usar a palavra reservada override para reimplementar, já em java basta você marcar o método na classe filha com o atributo “@Override”.

Concluindo tópico 1 – Conceitos fundamentais de orientação a objeto

- Nesse nosso estudo, vimos:
 - Informações importantes sobre os paradigmas de desenvolvimento;
 - Vimos também os 4 pilares da OO (Abstração, Encapsulamento, Herança e Polimorfismo);
 - Cada linguagem irá implementar esses pilares de uma forma, mas essencialmente é a mesma coisa.
 - Apenas a questão da herança, como comentado, que pode trazer variações mais bruscas, como a presença de herança múltipla.
 - Além disso, o encapsulamento também é feito de maneiras distintas nas diversas linguagens, embora os *getters* e *setters* sejam praticamente onipresentes.

Introdução tópico 2 – Conceitos fundamentais de análise e projetos OO

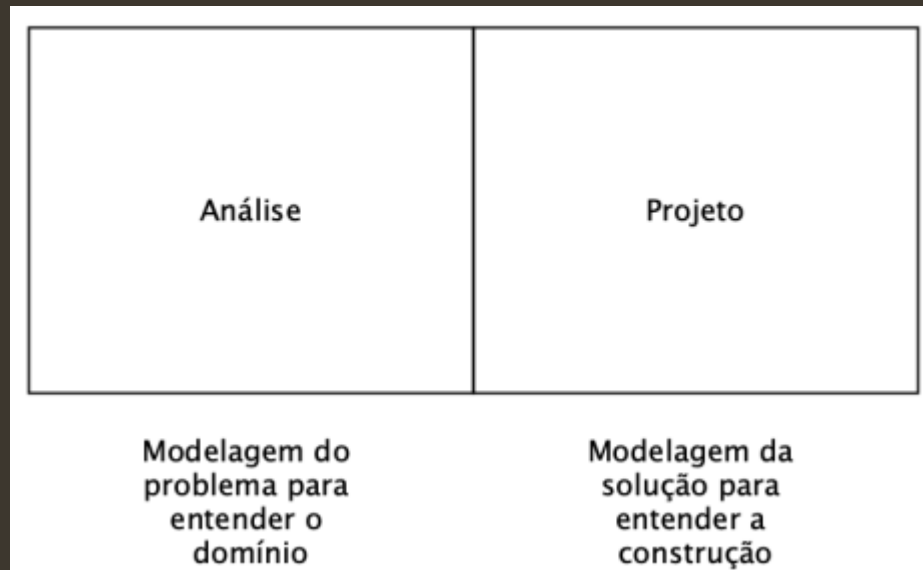
- Entendido os conceitos do que vem a ser Orientação a Objeto (OO), vamos analisar como utilizar os mesmos na Análise e Projetos Orientado a Objetos (APOO).

Conceitos fundamentais de análise e projetos OO

- Existem várias definições de APOO.
- Vamos considerar duas inicialmente:
 - Uma do ponto de vista de fases no ciclo de vida de desenvolvimento de software;
 - Outra do ponto de vista de partes interessadas no desenvolvimento de software.

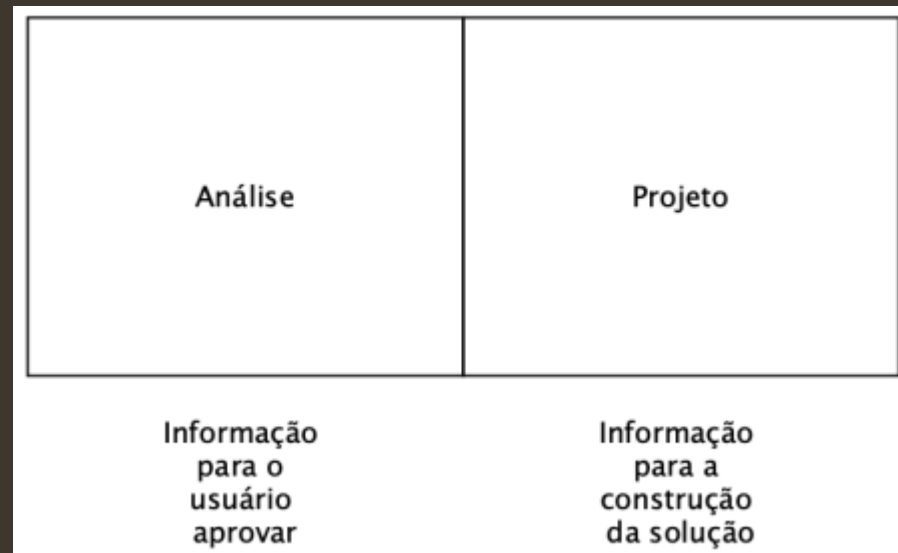
Conceitos fundamentais de análise e projetos OO

- Fases no ciclo de vida de desenvolvimento de software;
 - A análise modela o problema, com atividades que buscam entender o domínio do problema (o que deve ser feito) (atividade de investigação);
 - O projeto modela a solução, com atividades de criação (como deve ser feito) (uma atividade de engenharia).



Conceitos fundamentais de análise e projetos OO

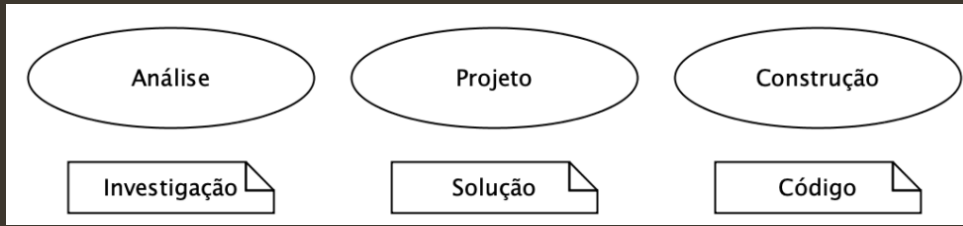
- Partes interessadas no desenvolvimento de software.
 - A análise: Todas as atividades necessárias para o conhecimento do usuário. Informações levantadas devem ser discutidas e aprovadas.
 - O projeto: As atividades que interessam apenas o pessoal que vai construir a solução



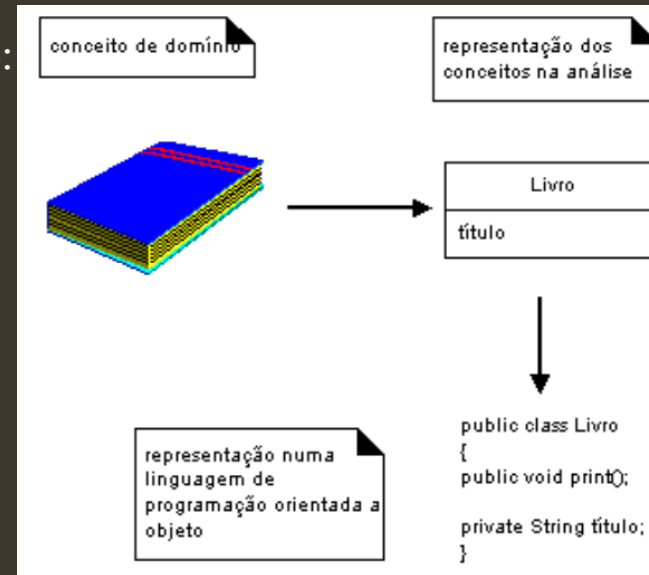
Conceitos fundamentais de análise e projetos OO

- Perceba que uma coisa não exclui a outra (análise ou projeto, e sim análise e projeto);
- Em nossas aulas vamos trabalhar a visão baseada no segundo ponto, pois queremos associar as palavras "análise" e "projeto" aos artefatos entregues nos final de cada fase

Conceitos fundamentais de análise e projetos OO



- Durante o desenvolvimento de uma solução segundo APOO nos deparamos com situações, como: em um sistema de biblioteca, como dividir a análise do projeto?
 - Análise: Alguns dos conceitos: livro, biblioteca, usuário... (que características e funções executam);
 - Projeto: Quais são os objetos lógicos que mapearão esses conceitos em uma linguagem OO?
- Baseado nisso, é importante observar algumas questões, como:
 - Não é verdade que haja correspondência 1-para-1 entre entidades no modelo de análise e entidades no modelo de projeto
 - Pode haver entidade do modelo de análise que não será representado no Projeto (é raro)
 - Pode haver entidade adicional no projeto (é frequente), por exemplo: Conexão de banco de dados, objeto controlador, cache de objetos, etc.



Concluindo tópico 2 – Conceitos fundamentais de análise e projetos OO

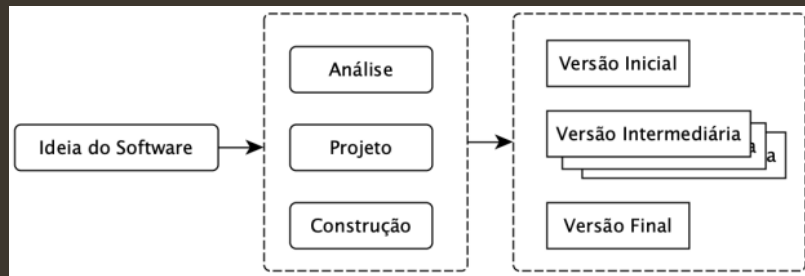
- Nesse nosso estudo, vimos:
 - Que análise e projeto OO não são excludentes e sim complementares
 - Vimos também, por meio de um exemplo, como isso é mapeado.

Introdução tópico 3 – Modelos iterativo, incremental e ágil

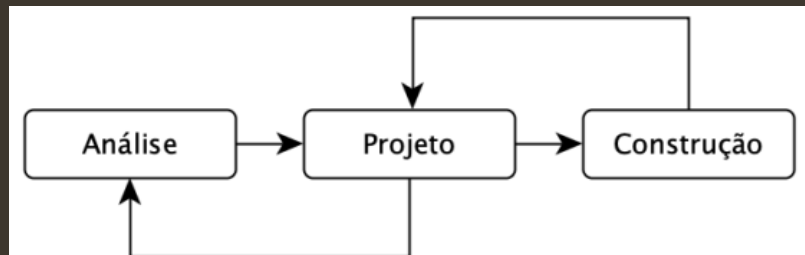
- Nesse estudo vamos entender o que vem a ser o modelo iterativo e o incremental e sua interconexão como o ágil para APOO.

Modelos iterativo, incremental e ágil

- Modelo incremental
 - No modelo de processo de software incremental tem-se a ideia de se desenvolver uma implementação inicial, apresentá-la ao usuário e evoluir o software ao longo do tempo de acordo com o feedback do usuário

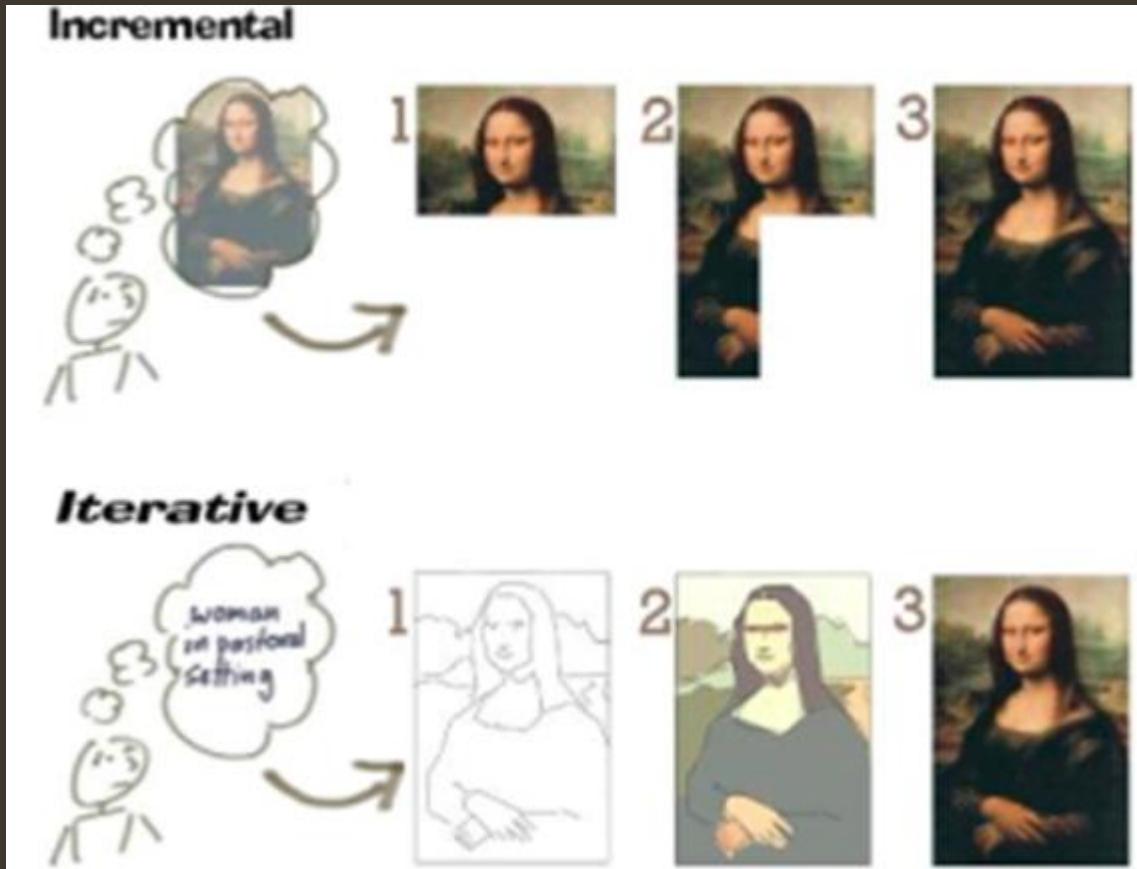


- No modelo iterativo, a cada iteração uma ou mais atividades são executadas antes de se iniciar uma nova atividade



Modelos iterativo, incremental e ágil

- Modelo interativo e incremental



Modelos iterativo, incremental e ágil

- Escultor interativo
 - Escolhe uma peça
 - Delineia as formas a cada interação
 - Cada vez fica mais perto do resultado final que ele projetou
 - A obra fica pronta como um todo.
- Escultor incremental
 - Escolhe uma peça
 - Faz cada parte por completo por vez (boca, tronco, pernas...)
 - A cada incremento a obra fica mais perto do todo;
 - A obra fica pronta aos pedaços.

Modelos iterativo, incremental e ágil

- Login interativo
 - Talvez a primeira interação é um protótipo;
 - A segunda o login com usuário e senha;
 - A terceira o login com usuário/cpf e senha;
 - A quarta o login com usuário/cpf/email e senha;
 - A quinta o login com as opções anteriores + acesso via rede social.
- Login incremental
 - O processo de login é feito de uma vez e quando entregue, outra *feature* é implementada.

Modelos iterativo, incremental e ágil

- Tela de pesq. interativo
 - Talvez a primeira interação é um protótipo;
 - Na segunda teremos campos de filtros primários;
 - Na terceira campos mais detalhados;
 - A quarta teria exportação para o excel...
- Tela pesq. incremental
 - A tela de pesquisa é feita de uma só vez, independente do sistema ter “n” *features* – o foco é ela.

Modelos iterativo, incremental e ágil

- Qual o melhor modelo?
 - É o uso em conjunto
 - E aqui entram os métodos ágeis
- Exemplo: o Scrum é iterativo e incremental
 - Interativo quando a cada Sprint (interação) quanto as implementações;
 - Isso pode ser dentro da Sprint (parte ou total)
 - Incremental quando a cada Sprint, ou sub conjunto, um pedaço do sw fica inteiramente pronto
 - Isso pode ser dentro da própria Sprint.

Concluindo tópico 3 – Modelos iterativo, incremental e ágil

- Nesse nosso estudo nós pudemos verificar mais sobre o uso dos modelos interativos e incrementais, seus pontos fortes e de melhoria.
- Vimos também como tirar o máximo proveito dos dois métodos por meio do uso dos métodos ágeis.

Conclusão da unidade I

- Até aqui em nosso estudo nós vimos:
 - Sobre os conceitos fundamentais de orientação a objeto
 - Abstração, Encapsulamento, Herança e Polimorfismo;
 - Sobre os conceitos fundamentais de análise e projetos orientado a objeto
 - Análise, Projeto e seu mapeamento
 - Sobre os modelos iterativo, incremental e ágil

Introdução da unidade II

- Como objetivo geral esta unidade iremos:
 - Aprender a etapa de concepção do projeto orientado a objetos
 - Aprender como gerenciar requisitos evolutivos
 - Aprender como gerenciar casos de uso

Introdução do tópico 1 – Introdução e Concepção

- Qual o cenário atual no qual projetos têm sido inseridos e como lidar com eles? Vamos entender um pouco mais...

Introdução e Concepção

- Vivemos num mundo complexo marcado por transformações constantes
- <> tipos de mindset – tomar decisões rápidas
- Dinâmica atual – acrônimo – mundo “VUCA”
 - volatility, uncertainty, complexity e ambiguity (volatilidade, incerteza, complexidade e ambiguidade).
 - Volatilidade: Para alcançar 50 MI usuários - Rádio 38 anos; TV 13 anos; Facebook 3,5 anos; Twitter 9 meses; Instagram 6 meses; Pokemon 22 dias.
 - Incerteza: muitas informações, muitas mudanças, muito mais difícil prever o amanhã (bolsa, direção das empresas)...
 - Complexidade: Está cada vez mais difícil explicar o mundo de forma linear (mudança aqui, impacta lá...)
 - Ambiguidade: Inúmeras interpretações para as mesmas coisas – dicotomias estão sendo questionadas.
 - O mundo VUCA força a busca por: Soluções criativas, inovadoras e ágeis.

Introdução e Concepção

- Neste contexto:
 - pressões por resultados rápidos; aquisições de tecnologia força que etapas iniciais do ciclo de vida para a realização de um projeto sejam negligenciadas.
 - Diante dessa crescente complexidade, tornou-se de vital importância a redução do risco associado às etapas de um projeto.
 - Faz-se necessária uma análise minuciosa do seu ciclo de vida, que compreende as etapas de concepção, requisitos, arquitetura, desenvolvimento/produção e operação, até a sua retirada de serviço/descarte.
 - Trata-se de uma abordagem holística, com base em uma visão de alto nível, que permite implementar e analisar sistemas compostos por elementos heterogêneos, complexos e independentes, para que se integrem de forma cooperativa e eficaz é necessário ter especial foco nas etapas iniciais de concepção e resistir a se ater exclusivamente à etapa de desenvolvimento e produção

Introdução e Concepção

- ## Concepção

- Como nasce, ou deveria nascer um projeto? Deve(ria) com planejamento ;-)
 - Momento 1: Durante o início do projeto
 - Qual seu valor de negócio? (vai reduzir custo? Vai aumentar receita?)
 - Ocorre a análise de viabilidade
 - Técnica (Podemos construir?)
 - Econômica (Vai fornecer valor ao negócio?)
 - Organizacional (Será usado?)
 - Comissão de aprovação decide se o projeto deve ser empreendido;

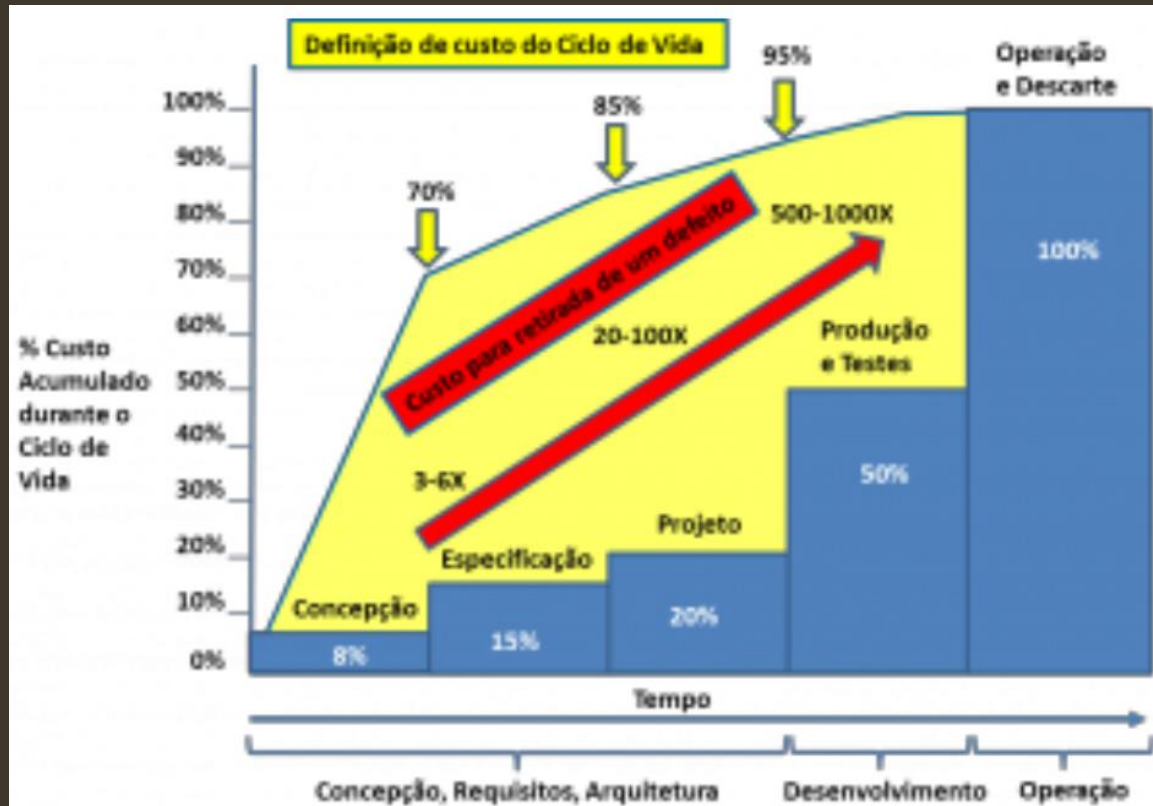
Introdução e Concepção

- ## Concepção

- Como nasce, ou deveria nascer um projeto? Deve(ria) com planejamento ;-)
 - Momento 2: Entra em gerenciamento de projeto
 - GP cria o plano do projeto
 - Contendo o plano de trabalho a ser executado
 - Contendo a equipe que fará parte do projeto

Introdução e Concepção

- Importância da fase de planejamento – DoD.



Introdução e Concepção

- O que vemos nesse gráfico?
 - A etapa de concepção, requisitos e arquitetura consome 20% de todo tempo acumulado de um projeto;
 - Mas elas são responsáveis por:
 - Concepção: 70% dos custos de um projeto;
 - Requisito e Arquitetura: 25%;
 - Ao todo, 95% do custo de um projeto são impactados pelo trabalho definido nessa fase inicial de um projeto
 - Esse estudo confirma outros realizados que apontam para que um problema em produção pode ter um custo de correção de 500 a 1000 vezes maior do que se fosse detectado na fase de concepção.

Conclusão do tópico 1 - Introdução e Concepção

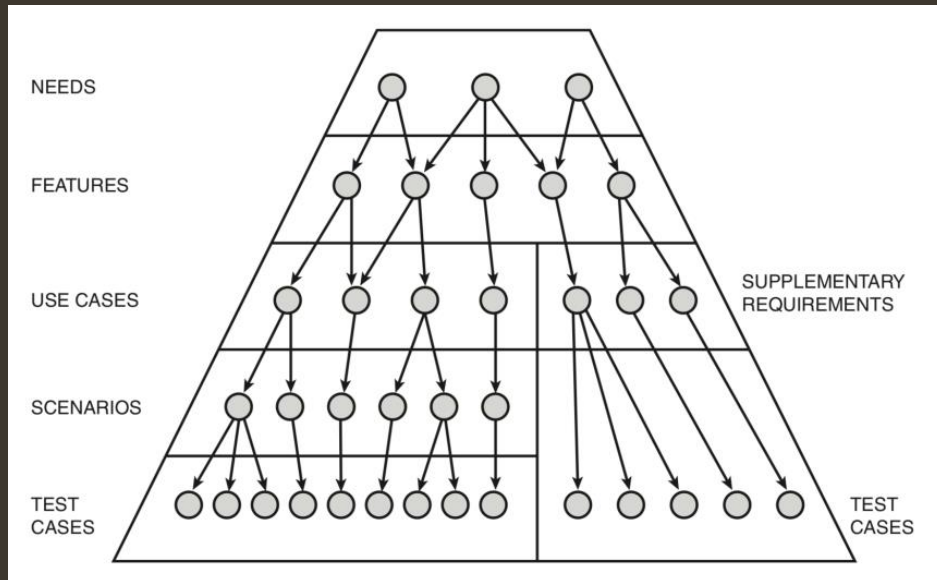
- Pudemos observar nesse estudo que
 - Estamos vivendo em um ambiente altamente complexo e de muitas modificações;
 - Essa informação deve reforçar que precisamos ser ágeis, mas não podemos negligenciar aquilo que já sabemos, por meio de estudos, que é o certo a fazer;
 - Ir direto para a codificação e entrega é o mesmo que cortar a árvore sem amolar o machado

Introdução tópico 2 – Requisitos evolutivos

- Nesse estudo veremos que Software tem uma característica diferente de outros produtos de engenharia: seus requisitos são evolutivos. Vamos entender isso no decorrer deste estudo.

Requisitos evolutivos

- Antes de começar, vamos entender o que vem a ser um Requisito
 - Uma das definições aceitas é que um requisito é uma condição ou capacidade que um software deve atender (LARMAN, 2007)
 - É importante observar que existem os requisitos que não são funcionais, ou seja, os não funcionais;



Requisitos evolutivos

- Necessidades de usuários: requisitos que são demandados por usuários ou especialistas que conhecem do domínio do problema que o software deve atender
- Funcionalidade: serviços que devem ser fornecidos pelo software para os usuários
- Casos de uso: descrição do comportamento de uma funcionalidade em uma sequencia de passos
- Cenários de casos de uso: maneiras de utilização dos casos de uso desenvolvidos
- Requisitos suplementares: esse é um outro nome para requisitos não funcionais
- Casos de testes: especificação de cenários de testes com os resultados esperados para cada cenário de caso de uso

Requisitos evolutivos

- Existem algumas características que devem ser observadas para obter um requisito de boa qualidade
 - Não ambíguo: isso quer dizer que os requisitos não podem ter dupla implementação pois isso dificulta o entendimento e a comunicação dos mesmo
 - Testável: um requisito precisa ser testado para que o mesmo possa ser implementado e testado de maneira objetiva
 - Claro: não basta ser não ambíguo, ele precisa ser de simples entendimento, sem a utilização de recursos de linguagem desnecessários
 - Correto: o requisito deve expressar a necessidade dos usuários da maneira mais próxima possível da realidade
 - Viável: o requisito precisa ser possível ser implementando com os recursos financeiros, de tempo, e demais existentes
 - Livre de tecnologia: os requisitos inicialmente não devem se ater a questões tecnológicas (exceto por quesitos de viabilidade) para não limitar a solução de maneira desnecessária
 - Necessário: requisitos tem um custo para serem desenvolvidos, dessa forma, os mesmo devem ser considerados como necessários para serem implementados
 - Contido: um requisito deve realizar uma única função de maneira mais independente possível

Conclusão do tópico 2 - Requisitos evolutivos

- Nesse estudo vimos que um requisito evolui a medida que o entendimento da necessidade se tornar mais claro e vimos como é feito isso assim de como aferir se é de boa qualidade.

Introdução do tópico 3 – Casos de Uso

- Nesse estudo iremos verificar os casos de uso são uma parte muito importante da análise orientada a objeto (AOO). Se não for o mais importante!

Casos de Uso

- Definindo caso de uso:
 - Na Engenharia de Software, um caso de uso (do inglês use case) é um tipo de classificador representando uma unidade funcional coerente provida pelo sistema, subsistema, ou classe manifestada por sequências de mensagens intercambiáveis entre os sistemas e um ou mais atores (LARMAN, 2007)
- Os casos de uso são compostos de dois elementos básicos: (a) o diagrama de caso de uso e o (b) descritivo de caso de uso.

Casos de Uso

- Diagrama de caso de uso
 - Tem o objetivo de descrever os requisitos funcionais do sistema e sua relação com atores.
- É composto por:
 - Ator
 - Caso de uso
 - Associação

Casos de Uso

- Descritivo de caso de uso
 - O descritivo de um caso de uso apresenta passo a passo como o mesmo deve ser executado
- Geralmente composto por:
 - **Nome:** Identificador inequívoco do caso de uso, deve ser escrito em formato de verbo/substantivo e ser suficiente para o utilizador perceber a que se refere o caso de uso.
 - **Iteração ou estado de desenvolvimento:** Descrição do estado atual do caso de uso à medida que este vai evoluindo.
 - **Sumário:** Curto resumo do caso de uso.
 - **Pré-condições:** Listagem das condições que se devem verificar quando o utilizador inicia este caso de uso. Não incluem triggers.
 - **Triggers:** Eventos que ocorrem dando início ao caso de uso. Podem ser externos, temporais ou internos.

Casos de Uso

- **Linha de Eventos:** Esta secção descreve o curso de eventos ou cenário que se realiza. Usualmente é descrito através de uma sequência de eventos numerados.
- **Percursos Alternativos:** Descrição de percursos alternativos à linha de eventos básica.
- **Pós-condições:** Descrição do estado do sistema após a execução do caso de uso
- **Regras de negócio:** Secção reservada para informação adicional relativa à política da empresa ou restrições impostas pelo tipo de negócio.
- **Notas:** Informação adicional relativamente ao caso de uso, não coberta pelas secções anteriores.
- **Autor e data:** Listagem dos autores e datas das várias versões revistas.

Nome: Pesquisar Online

Descrição: O pesquisador realiza N pesquisas com uma string de consulta

Ator: Pesquisador

Pré-condições: String de consulta válida (RN01)

Fluxo Principal

Fluxo Alternativo

Fluxo Exceção

Pós-condições: Exibir os resultados da consulta realizada

Regras: RN02, RN04

RN01 - String de consulta válida

Se
 (String.valor != nulo E String.valor != espaço)
Então
 Válida
Senão
 Inválida

Conclusão do tópico 3 - Casos de Uso

- Neste estudo entendemos como as necessidades dos usuários podem ser decompostas até os casos de uso. Entendemos também como os casos de uso permeiam o desenvolvimento de software durante a fase de projeto. Além disso, vimos com detalhes como funciona o diagrama de casos de uso e como funciona o descritivo de caso de uso. Lembrando um ponto importante que vem a ser as regras de negócio que devem ser separadas e baseadas em IF-THEN

Conclusão geral da unidade - II

- Vimos nesse estudo elementos relevantes quanto concepção de um projeto, vimos em que cenário estamos inseridos no dia-a-dia (mundo VUCA), também passamos sobre os elementos ligados à requisitos e caso de uso. Todos esses elementos fundamentais para uma correta AOO.

Introdução da unidade 3

- Objetivos de aprendizagem
 - Aprender sobre o diagrama de classes
 - Aprender sobre o diagrama de atividades
 - Aprender sobre o diagrama de maquina de estados

Introdução do tópico 1 – Diagrama de Classe

- Nesse estudo iremos avaliar como o diagrama de classes pode nos ajudar no estudo da solução proposta para o projeto.

Diagrama de Classe

- Um diagrama de classes na *Unified Modeling Language* (UML) é um tipo de diagrama estático que descreve a estrutura de um sistema mostrando as classes do sistema, seus atributos, operações (ou métodos) e as relações entre esses objetos
- O diagrama de classes é o principal bloco de construção da modelagem orientada a objetos

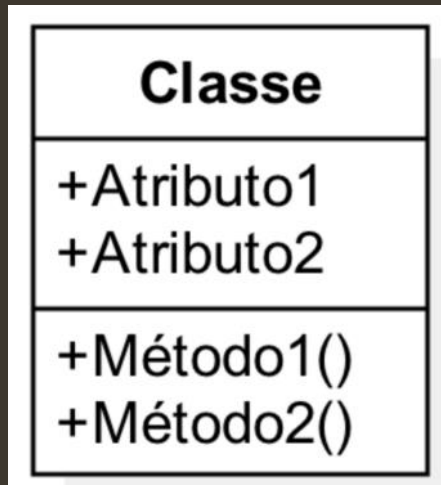


Diagrama de Classe

- Visibilidade
 - Para especificar a visibilidade de um membro da classe (ou seja, qualquer atributo ou método), essas notações devem ser colocadas antes do nome do membro:
 - + Público
 - - Privado
 - # Protegido
 - ~ Pacote

Diagrama de Classe

- Relacionamentos
 - Forma como classes e objetos se integram
 - Relacionamento em nível de instância
 - Dependência
 - Associação
 - Agregação
 - Composição
 - Relacionamento em nível de classe
 - Generalização / Herança
 - Multiplicidade

Conclusão do tópico 1 – Diagrama de Classe

- Nesse estudo pudemos verificar o que é o diagrama de classe, o que ele representa a nível de proposta de solução e quais são seus principais componentes.

Introdução do tópico 2 – Diagrama de Atividades

- Nesse estudo iremos avaliar o diagrama de atividades e como sua proposta de modelagem de processo computacional e organizacional pode nos ajudar na modelagem de solução do problema ao qual o software se propõe.

Diagrama de Atividades

- Construção
 - Sua construção segue o seguinte conjunto de componentes:
 - Elipses representam ações
 - Diamantes representam decisões
 - Barras representam o início (divisão) ou final (união) de atividades simultâneas
 - Um círculo preto representa o início (nó inicial) do fluxo de trabalho
 - Um círculo preto cercado representa o final (nó final)
 - As setas correm desde o início até o final e representam a ordem em que as atividades acontecem

Diagrama de Atividades

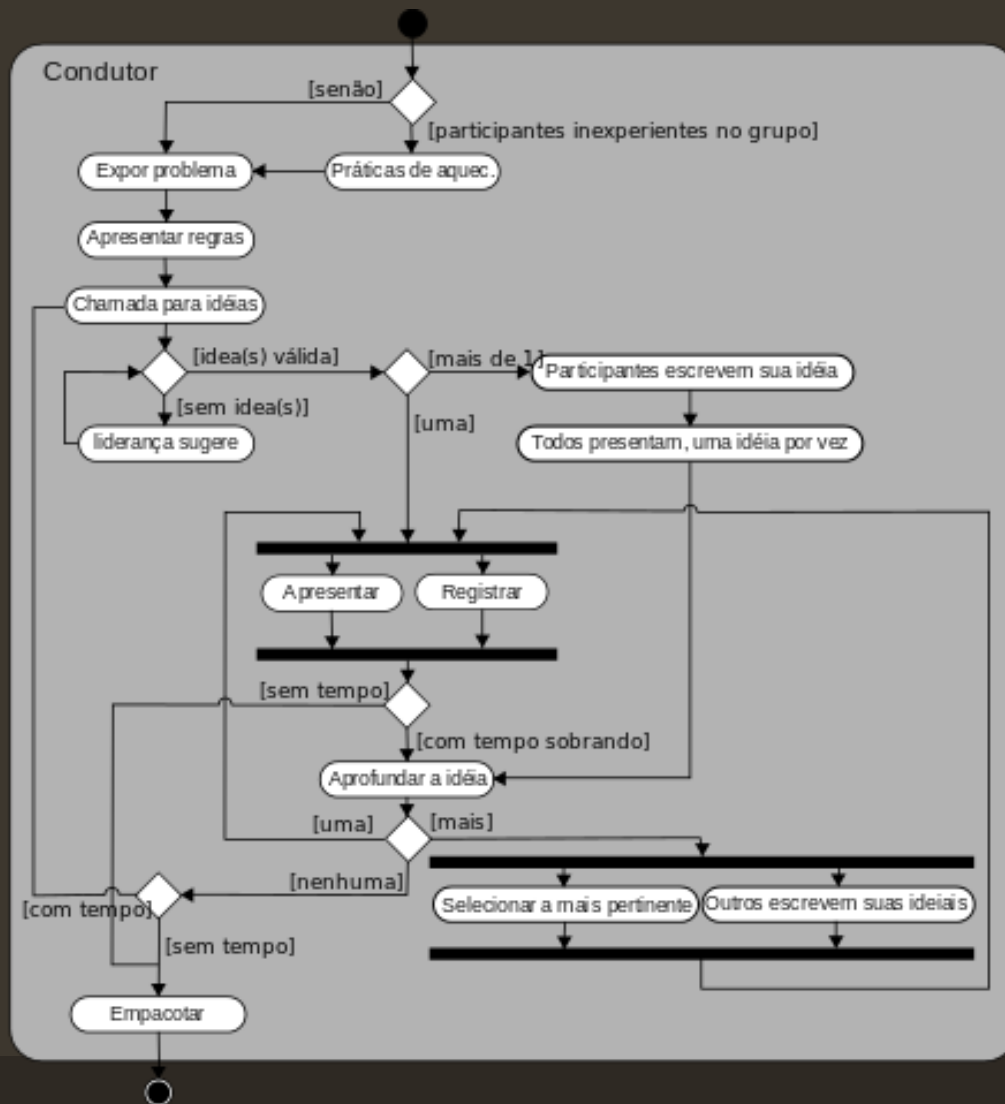


Diagrama de Atividades

- Os diagramas de atividades podem ser considerados como uma forma de fluxograma estruturado combinado com um diagrama de fluxo de dados tradicional.
- As técnicas típicas de fluxograma não possuem construções para expressar a simultaneidade.
- No entanto, os símbolos de junção e divisão nos diagramas de atividade resolvem isso apenas para casos simples; o significado do modelo não é claro quando eles são combinados arbitrariamente com decisões ou loops

Diagrama de Atividades

- Enquanto na UML 1.x, os diagramas de atividades eram uma forma especializada de diagramas de estados, na UML 2.x, os diagramas de atividades foram reformuladas para serem baseados na semântica semelhante à rede de Petri, aumentando o escopo de situações que podem ser modeladas usando diagramas de atividades.
- Essas alterações fazem com que muitos diagramas de atividades da UML 1.x sejam interpretados de maneira diferente na UML 2.x (PENDER, 2004). Os diagramas de atividade UML na versão 2.x podem ser usados em vários domínios, por exemplo, no design de sistemas embarcados. É possível verificar essa especificação usando a técnica de verificação de modelos.

Conclusão do tópico 2

- Vimos nesse estudo a importância do diagrama de atividade assim como sua utilização, mas é importante deixar claro a diferença de escopo conforme a versão da modelagem UML que se esteja usando (1.x ou 2.x).

Introdução do tópico 3

- Nesse estudo iremos avaliar o diagrama de máquina de estados, entender seu funcionamento e como ele pode ajudar na resolução do problema de sw proposto.

Diagrama de Máquina de Estados

- A máquina de estados UML, também conhecida como *statechart* UML, é uma realização significativamente aprimorada do conceito matemático de um autômato finito em aplicações de ciência da computação, conforme expresso na notação UML
- Foco é que cada programa, processo, componente esteja exatamente em um estado conhecido e haja entendimento/controle sobre como essas transições serão realizadas.
- Os estados de uma entidade são muito importantes para definir onde inicia o ciclo de vida e onde termina

Diagrama de Máquina de Estados

- Muitos sistemas de software são orientados a eventos, o que significa que eles esperam continuamente pela ocorrência de algum evento externo ou interno, como um clique do mouse, um pressionamento de botão, um registro de tempo ou a chegada de um pacote de dados.
- Depois de reconhecer o evento, esses sistemas reagem executando a computação apropriada que pode incluir a manipulação do hardware ou a geração de eventos “*soft*” que acionam outros componentes de software internos.
- (Por isso, os sistemas acionados por eventos são alternativamente chamados de sistemas reativos.) Quando o tratamento de eventos é concluído, o sistema volta a esperar pelo próximo evento
- A resposta a um evento geralmente depende tanto do tipo do evento quanto do estado interno do sistema e pode incluir uma mudança de estado levando a uma transição de estado. O padrão de eventos, estados e transições de estado entre esses estados pode ser abstraído e representado como uma máquina de estado finito (FSM)

Diagrama de Máquina de Estados

- **Diagramas básicos de estado UML**

- A UML preserva a forma geral dos diagramas de estado tradicionais. Os diagramas de estado da UML são gráficos direcionados nos quais os nós denotam estados e conectores denotam transições de estado.

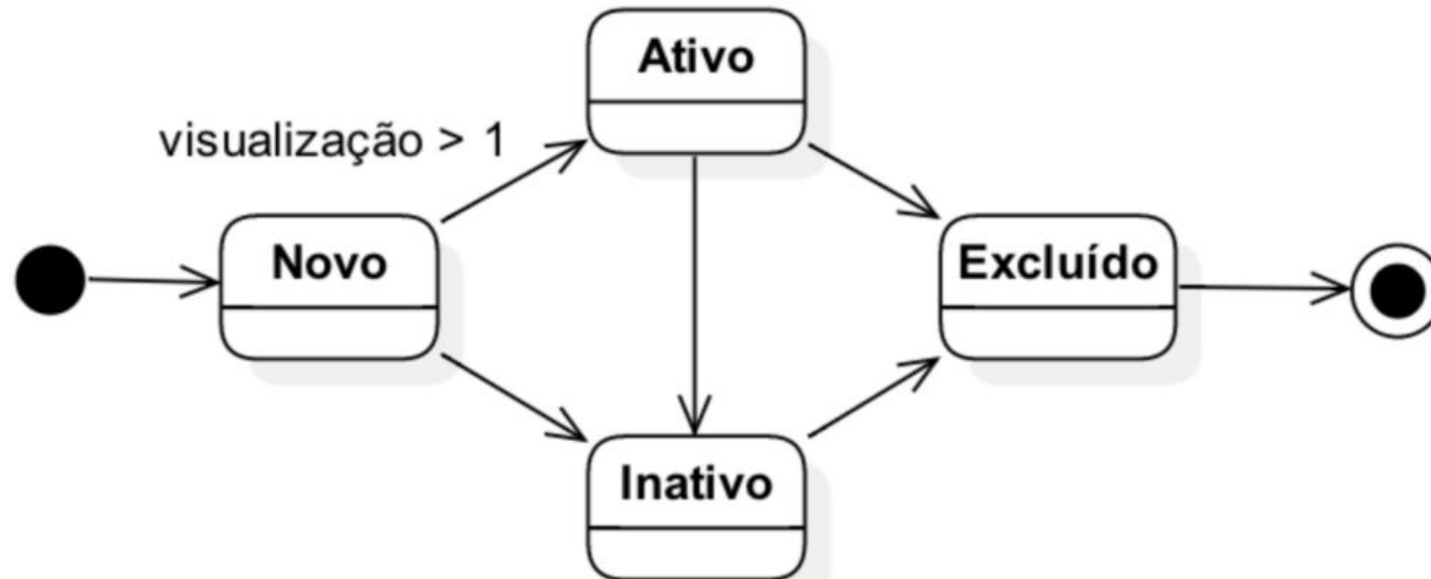


Diagrama de Máquina de Estados

- **Diagramas básicos de estado UML**
 - As transições, representadas como setas, são rotuladas com os eventos desencadeadores seguidos, opcionalmente, pela lista de ações executadas. A transição inicial se origina do círculo sólido e especifica o estado padrão quando o sistema é iniciado pela primeira vez. Cada diagrama de estado deve ter essa transição, que não deve ser rotulada, pois não é acionada por um evento. A transição inicial pode ter ações associadas.
- **Eventos**
 - Um evento é algo que acontece e afeta o sistema
- **Estados**
 - Geralmente controlado por variáveis (enum). Assim, uma variável de estado (e modelo clássico de FSM) é que o valor da variável de estado define completamente o estado atual do sistema em qualquer momento
- **Condições de guarda**
 - As condições de guarda estão associadas com regras de negócio. Pense em termos do ponto de vista do negócio ao defini-las
 - Transições e ações serão tomadas quando elas forem TRUE

Diagrama de Máquina de Estados

- **Ações e transições**
 - Quando uma instância de evento é despachada, a máquina de estado responde executando ações, como alterar uma variável, executar E/S, chamar uma função, gerar outra instância de evento ou alterar para outro estado. Quaisquer valores de parâmetro associados ao evento atual estão disponíveis para todas as ações causadas diretamente por esse evento. A mudança de um estado para outro é chamada de transição de estado, e o evento que a causa é chamado de evento desencadeador ou simplesmente o acionador

Conclusão do tópico 3 – Diagrama de Máquina de Estados

- Vimos nesse estudo a importância do diagrama de máquina de estados e avaliamos seus componentes e como usá-los.

Conclusão da unidade III

- Nessa unidade vimos:
 - Sobre o que é um projeto OO e quais alguns dos diagramas que podemos utilizar para a construção do mesmo. Esses diagramas são os fundamentos da UML e representam a estrutura de dados, via diagrama de classes, a sequencia de atividades, via diagrama de atividades e os estados possíveis que uma entidade pode assumir, via maquina de estados. Esses diagramas tem uma função importante para realizar um projeto OO permitindo ao desenvolvedor entender a solução do problema por alguns pontos de vista diferentes.

Introdução da unidade 4

- Objetivos de aprendizagem (Padrões comportamentais)
 - Aprender sobre a separação de responsabilidades
 - Aprender sobre o *Domain Driven Development*
 - Aprender sobre BPMN, DMN e CMN

Introdução do tópico 1 – Separação de Responsabilidades

- Vimos o básico da construção de um projeto orientado a objetos.
- No entanto, existem diversas técnicas que podem ser exploradas para melhoria do resultado final, sendo uma análise de melhor qualidade ou um projeto de melhor qualidade.
- O importante é saber a técnica e como utilizá-la da melhor forma possível. Para isso, vamos estudar uma notação muito útil para a diagramação de processos chamada BPMN. Vamos estudar também a notação DMN que serve para a criação de regras baseadas em *IF-THEN*. E por fim, vamos estudar a notação CMN que serve para representação de casos. Tudo isso associado à separação de responsabilidades e o desenvolvimento orientado ao domínio.

Separação de Responsabilidades

- Na ciência da computação, a separação de responsabilidades (SoC) é um princípio de design para separar um programa de computador em seções distintas, de modo que cada seção aborda uma preocupação separada.
- Uma preocupação é um conjunto de informações que afeta o código de um programa de computador. Uma preocupação pode ser tão geral quanto os detalhes do hardware para o qual o código está sendo otimizado, ou tão específico quanto o nome de uma classe para instanciar.
- Um programa que incorpora SoC bem é chamado de programa modular
- Projetos em camadas em sistemas de informação são outra forma de realização da separação de interesses (por exemplo, camada de apresentação, camada de lógica de negócios, camada de acesso a dados, camada de persistência)
- Quando as preocupações são bem separadas, há graus mais altos de liberdade para reutilização de módulos, bem como desenvolvimento e atualização independentes

Separação de Responsabilidades

- Implementação
 - Os mecanismos para programação modular ou orientada a objetos que são fornecidos por uma linguagem de programação são mecanismos que permitem aos desenvolvedores fornecer SoC.
 - Por exemplo, linguagens de programação orientadas a objeto, como C #, C ++, Delphi e Java podem separar preocupações em objetos, e padrões de design de arquitetura como MVC ou MVP podem separar conteúdo de apresentação e processamento de dados (modelo) de conteúdo.
 - O design orientado a serviços pode separar as preocupações em serviços.
 - Linguagens procedurais de programação, como C e Pascal, podem separar preocupações em procedimentos ou funções.
 - Linguagens de programação orientadas a aspectos podem separar preocupações em aspectos e objetos

Separação de Responsabilidades

- Implementação
 - Separação de preocupações é um princípio de design importante em muitas outras áreas, bem como, planejamento urbano, arquitetura e design de informação;
 - Exemplos separar um espaço em salas, de modo que a atividade em um cômodo não afete pessoas em outros cômodos, e manter o fogão em um circuito e as luzes em outro, de modo que a sobrecarga do fogão não apague as luzes.
 - O exemplo com circuitos demonstra que a atividade dentro de um módulo, que é um circuito com consumidores de eletricidade conectados, não afeta a atividade em um módulo diferente, então cada módulo não está preocupado com o que acontece no outro

Separação de Responsabilidades

- Origem
 - O termo separação de preocupações provavelmente foi cunhado por Edsger W. Dijkstra em seu artigo de 1974 "Sobre o papel do pensamento científico"
- Exemplos
 - **Pilha de protocolos da Internet**
 - **HTML, CSS, JavaScript**
 - **Programação orientada a aspectos**

Conclusão da unidade 1 – Separação de Responsabilidade

- Vimos nesse estudo itens relacionados aos conceitos de separação de responsabilidades, assim como exemplos de seu uso.

Introdução do tópico 2 – Domain Driven Development (DDD)

- Nesse estudo vamos avaliar o design orientado por domínio (DDD). Ele é uma abordagem para o desenvolvimento de software para necessidades complexas, conectando a implementação a um modelo em evolução.

Domain Driven Development (DDD)

- A premissa do design orientado a domínio é o seguinte:
 - Colocando o foco principal do projeto no domínio principal e na lógica do domínio;
 - Baseando desenhos complexos em um modelo do domínio;
 - Iniciar uma colaboração criativa entre especialistas técnicos e de domínio para refinar, de forma iterativa, um modelo conceitual que aborde problemas de domínio específicos

Domain Driven Development (DDD)

- Vou fazer uma pergunta : O que é mais importante em um software ?
 - O código e a tecnologia usadas , diria o desenvolvedor;
 - Os testes , diria o testador;
 - O plano de projeto , diria o gerente;
 - A arquitetura usada , diria o arquiteto;

Faltou alguma coisa? Sim, faltou!!! Faltou o cliente responder, que o mais importante é o software resolver o problema dele!

Conclusão: o mais importante é a regra de negócio.

O Domain-Driven Development (DDD) é uma abordagem de desenvolvimento de software que reúne um conjunto de conceitos, princípios e técnicas cujo foco está no domínio e na lógica do domínio com o objetivo de criar um Domain Model ou (*modelo do domínio*).

Domain Driven Development (DDD)

- Conceitos
 - Contexto: O cenário em que aparece uma palavra ou declaração que determina seu significado;
 - Domínio: Uma esfera de conhecimento (ontologia), influência ou atividade. A área de assunto para a qual o usuário aplica um programa é o domínio do software;
 - Modelo: Um sistema de abstrações que descreve aspectos selecionados de um domínio e pode ser usado para resolver problemas relacionados a esse domínio;
 - Linguagem Ubíqua: Uma linguagem estruturada em torno do modelo de domínio e usada por todos os membros da equipe para conectar todas as atividades da equipe ao software (faturamento, nota, data limite).

Domain Driven Development (DDD)

- Design dirigido por domínio estratégico
 - O Design Estratégico é um conjunto de princípios para manter a integridade do modelo, destilação do Modelo de Domínio e trabalhar com vários modelos.
- Contexto Limitado
 - Defina explicitamente o contexto no qual um modelo se aplica. Explicitamente estabeleça limites em termos de organização da equipe, uso em partes específicas do aplicativo e manifestações físicas, como bases de código e esquemas de banco de dados. Mantenha o modelo estritamente consistente dentro desses limites

Domain Driven Development (DDD)

- Integração Contínua
 - Quando várias pessoas estão trabalhando no mesmo contexto limitado, há uma forte tendência para o modelo se fragmentar.
 - Quanto maior a equipe, maior o problema
 - Instale um processo de mesclagem de todos os códigos e outros artefatos de implementação com frequência, com testes automatizados para sinalizar a fragmentação rapidamente
 - Exercer incansavelmente a linguagem onipresente para elaborar uma visão compartilhada do modelo à medida que os conceitos evoluem na cabeça de diferentes pessoas.

Domain Driven Development (DDD)

- Mapa de Contexto
 - Ter os contextos todos mapeados e entendimento de suas divisões
 - Entender seus limites
 - Cuidado para que as formas de comunicação entre esses contextos não se perca;
 - Manter cuidado sob os sistemas legados!

Conclusão do tópico 2 - Domain Driven Development (DDD)

- Vimos neste estudo informações sobre o que é o DDD, vimos suas características, componentes e cuidados que devemos ter quanto ao seu uso em projetos.

Introdução do tópico 3 – BPMN, DMN e CMMN

- Nesse estudo vamos avaliar o que são BPMN, DMN e CMMN e como fazer uso deles em nossos projetos.

BPMN

O *Business Process Model e Notation* (BPMN) é uma representação gráfica para especificar processos de negócios em um diagrama de processo

O BPMN foi desenvolvido pela *Business Process Management Initiative* (BPMI) e é mantido pelo *Object Management Group* (OMG) desde 2005;

A versão 2.0 do BPMN foi lançada em janeiro de 2011, ponto em que o nome foi adaptado para *Business Process Model e Notation* como semântica de execução também foram introduzidos ao lado dos elementos notacionais e de diagramação.

Embora seja uma especificação OMG, o BPMN também é ratificado como ISO 19510. A última versão é BPMN 2.0.2, publicada em janeiro de 2014.

BPMN

O BPMN é um padrão para modelagem de processos de negócios que fornece uma notação gráfica para especificar processos de negócios em um *Business Process Diagram* (BPD), baseado em uma técnica de fluxograma muito semelhante aos diagramas de atividades do *Unified Modeling Language* (UML).

O objetivo do BPMN é oferecer suporte ao gerenciamento de processos de negócios.

O BPMN serve como uma linguagem comum, colmatando a lacuna de comunicação que frequentemente ocorre entre o design e a implementação do processo de negócios.

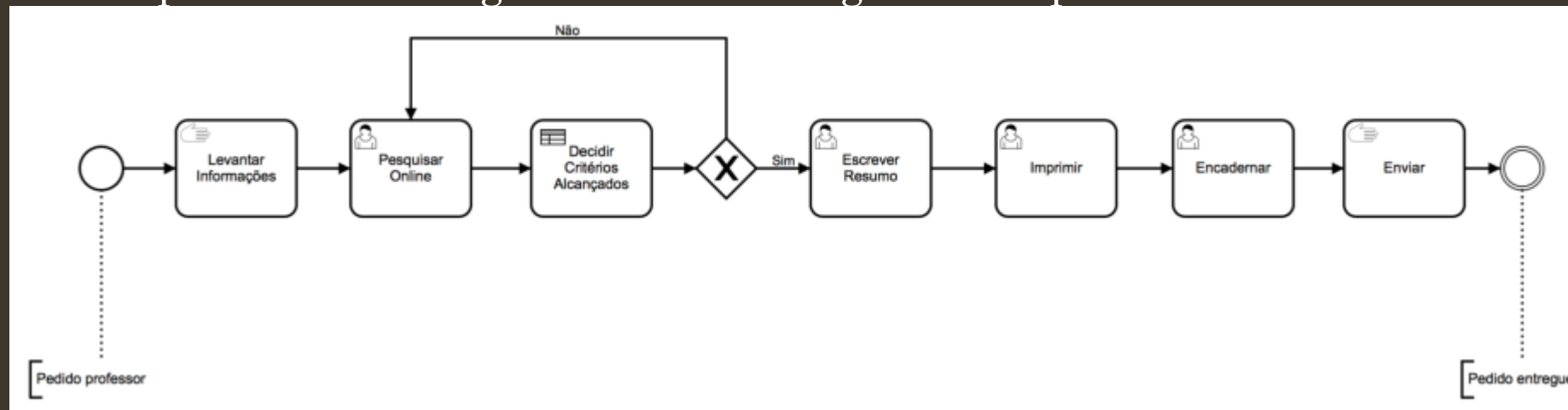
BPMN

As quatro categorias básicas de elementos do BPMN são:

- Objetos de fluxo
 - Eventos, atividades, gateways
- Conectando Objetos
 - Fluxo de sequência, fluxo de mensagens, associação
- Pistas de Natação
 - Piscina, pista
- Artefatos
 - Objeto de dados, grupo, anotação

BPMN

O *Business Process Model e Notation* (BPMN) é uma representação gráfica para especificar processos de negócios em um diagrama de processo



O BPMN foi complementado por dois novos padrões para a construção de modelos de gestão de casos e modelos de decisão, o *Case Management Model and Notation* (2013) e o *Decision Model and Notation* (2014).

DMN

É uma abordagem padrão para descrever e modelar decisões repetitivas dentro das organizações para garantir que os modelos de decisão sejam intercambiáveis entre as organizações

A notação é projetada para ser legível por usuários de negócios e de TI. Isso permite que vários grupos colaborem efetivamente na definição de um modelo de decisão

- Os empresários que gerenciam e monitoram as decisões
- Os analistas de negócios ou analistas funcionais que documentam os requisitos de decisão inicial e especificam os modelos de decisão detalhados e a lógica de decisão
- Os desenvolvedores técnicos responsáveis pela automação dos sistemas que tomam as decisões

DMN

O padrão DMN pode ser efetivamente usado de forma independente, mas também é complementar aos padrões BPMN e CMMN.

Dish				Show details
U	Input +	Output +	Annotation	
	Season	Dish		
1	"Fall"	"Spareribs"	-	
2	"Winter"	"Roastbeef"	-	
3	"Spring"	"Steak"	-	
4	"Summer"	"Light Salad and a nice Steak"	Hey, why not!?	

DMN

Esta é uma notação bastante simples definida no padrão DMN, que basicamente consiste em

- **Decisões:** O ato de determinar um valor de saída a partir de um número de valores de entrada, usando a definição lógica. Essa lógica de decisão é o que você pode expressar nas tabelas de decisão.
Dados de Entrada: Os dados de entrada que você "alimenta" em sua lógica de decisão para determinar o valor de saída.
Relações entre Decisões: Você pode conectar decisões com setas e, portanto, indicar qual saída de decisão será considerada como uma entrada para outra decisão.
Existem mais alguns símbolos na notação DRD, no entanto os mais relevantes são estes três.

CMMN

O *Case Management Model and Notation* (CMMN) é uma representação gráfica para expressar graficamente um caso, bem como um formato de troca para trocar modelos de Caso entre diferentes ferramentas

Um consórcio de 11 empresas contribuiu para o desenvolvimento do CMMN, que agora está sendo mantido pelo Grupo de Gerenciamento de Objetos.

A versão 1.0 do CMMN foi lançada em maio de 2014 e alterada pela versão 1.1 em dezembro de 2016

CMMN

CMMN ou BPMN?

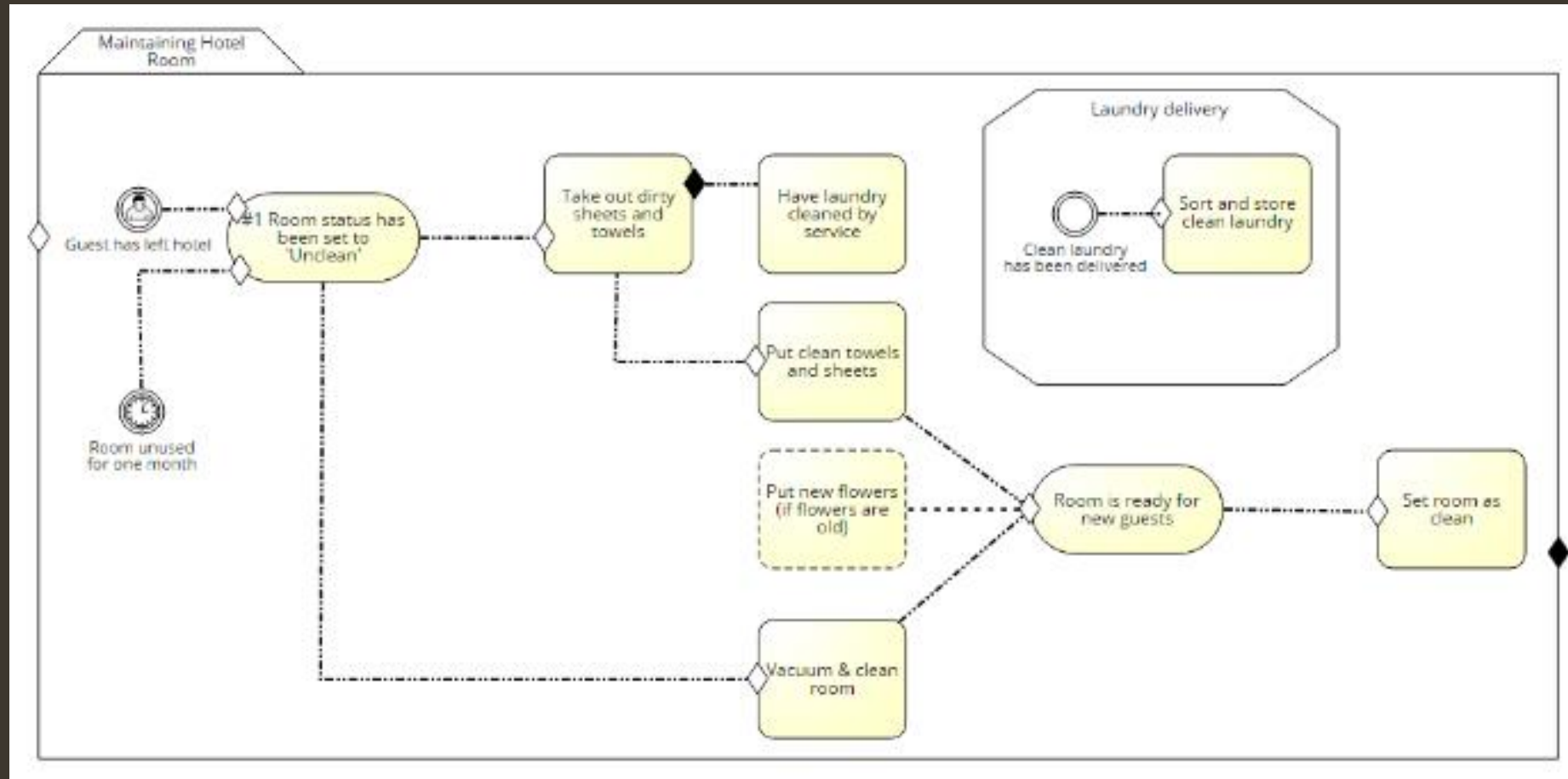
A resposta é ambos, depende do cenário que se precisa modelar;

Processos bem prescritivos – BPMN

Processos onde há necessidade de planejamento e tomada de decisão enquanto ocorre a execução – CMMN

Seu foco é apoiar processos imprevisíveis, que exige forte conhecimento e não tão fortemente estruturados

CMMN



Conclusão do tópico 3

- Nesse estudo vimos as notações BPMN, DMN e CMMN que auxiliam na criação de análise orientada a objeto suplementando a UML e deixando o resultado muito mais rico de entendimento.

Conclusão da unidade 4

- Neste estudo aprendemos sobre o que vem a ser a separação de responsabilidade e como sua utilização pode melhorar a qualidade do software a ser produzido.
- Estudamos também as questões relacionadas com o DDD e sua aplicação no desenvolvimento de software. Esse assunto tem uma área de pesquisa aquecida e desperta o interesse de diversos pesquisadores ao redor do mundo.
- Por fim, vimos as notações BPMN, DMN e CMMN que auxiliar na criação de análise orientada a objeto suplementando a UML e deixando o resultado muito mais interessante.

Introdução da Unidade 5 – Cenários de utilização

- Objetivos de aprendizagem
 - Cenários comuns de utilização de análise e projetos orientado a objetos
 - Um caminho para análise e projeto
 - Boas práticas de desenho

Introdução do tópico 1 – Cenários comuns de utilização de análise e projeto orientado a objetos

Vamos avaliar os cenários mais comuns de sua utilização na análise e projeto orientado a objeto.

Cenários comuns de utilização de análise e projeto orientado a objeto

- Começando pela análise
 - Devemos ter em mente a técnica principal que deve ser usada para entender os requisitos de um projeto é caso de uso
 - É natural que nas fases iniciais do projeto não tenhamos todas as informações necessárias para a análise do projeto, então devemos ter em mente usar o ciclo iterativo e incremental;
 - A cada iteração e incremento teremos uma versão mais completa e detalhada que a anterior.
 - Quais passos podemos dar para que o descrito na linha anterior aconteça?

Cenários comuns de utilização de análise e projeto orientado a objeto

1. Obtenha os atores envolvidos (hierarquia, personas....)
2. Obtenha os casos de uso (já temos os atores, agora o que cada um faz?) (temos mais pessoas usando o mesmo módulo?)(previsão para criação de módulos, pelo agrupamento das features, e perfil de acesso)
3. Escreva o diagrama de caso de uso com as funcionalidades mapeadas (o descritivo de caso de uso identificando as pré e pós condições, fluxo principal, alternativos e de exceção. Identificamos também as regras de negócio);
4. Escreva as regras de negócio no formato If-Then (para que as mesmas possam ser testadas em tempo de análise)
5. Verifique se os artefatos estão livres de erro quanto a escrita;
6. Valide se o que foi desenvolvido foi realmente o que precisava ter sido implementado
7. Repita esses passos o quanto for necessário;

Cenários comuns de utilização de análise e projeto orientado a objeto

1. Quando meus casos de uso estão prontos para serem implementados?
 1. Resposta nem sempre é simples
 1. Qual o seu tempo disponível? Nem sempre é possível ficar escovando os casos de uso até que os mesmos fiquem “perfeitos”.
 2. Qual é o seu orçamento? Na maior parte das vezes é restrito. Além disso, muitas vezes não temos a garantia de que a validação foi feita de maneira adequadas pois os usuários nem sempre sabem o que precisam. Vão descobrir ao utilizar o software criado.

Cenários comuns de utilização de análise e projeto orientado a objeto

- Agora sobre projeto
 - Identifique as entidades (tendo os casos de uso conseguimos identificar as entidades envolvidas, mapear seus atributos e suas ações – para isso use o diagrama de classes)
 - Identifique o ciclo de vida das entidades (uma vez que temos as entidades, como elas mudam de estado? Isso pode ser respondido por um diagrama de transição de estados – faça para as classes mais importantes, ao menos).
 - Segmente as regras (como fará escrita das mudanças do comportamento dos objetos? Pense aqui em padrões de projetos e terá menos acoplamento em seus sistemas)
 - Verifique a consistência entre classe e estados (diagramas)
 - Valide (com base na tecnologia que o time vai usar para desenvolver, é viável fazer o que é proposto acima?)
 - Repita esses passos quantas vezes precisar.

Cenários comuns de utilização de análise e projeto orientado a objeto

- Onde colocar a regra de negócio? Nos métodos das entidades? Nas classes auxiliares? Na interface de usuário? Nas rotinas de banco de dados? Em um middleware? Isso é uma questão de projeto, mais especificamente de arquitetura.
- Duplicidade de código: independente de onde a regra for colocada, não deve existir duplicidade de código. Se isso acontecer a manutenção ficará prejudicada pois ao alterar em um local pode-se esquecer de alterar em outro.
- Testes: as regras devem ser bem testadas e dessa forma, se não forem bem separadas os testes ficarão prejudicados.

Conclusão do tópico 1

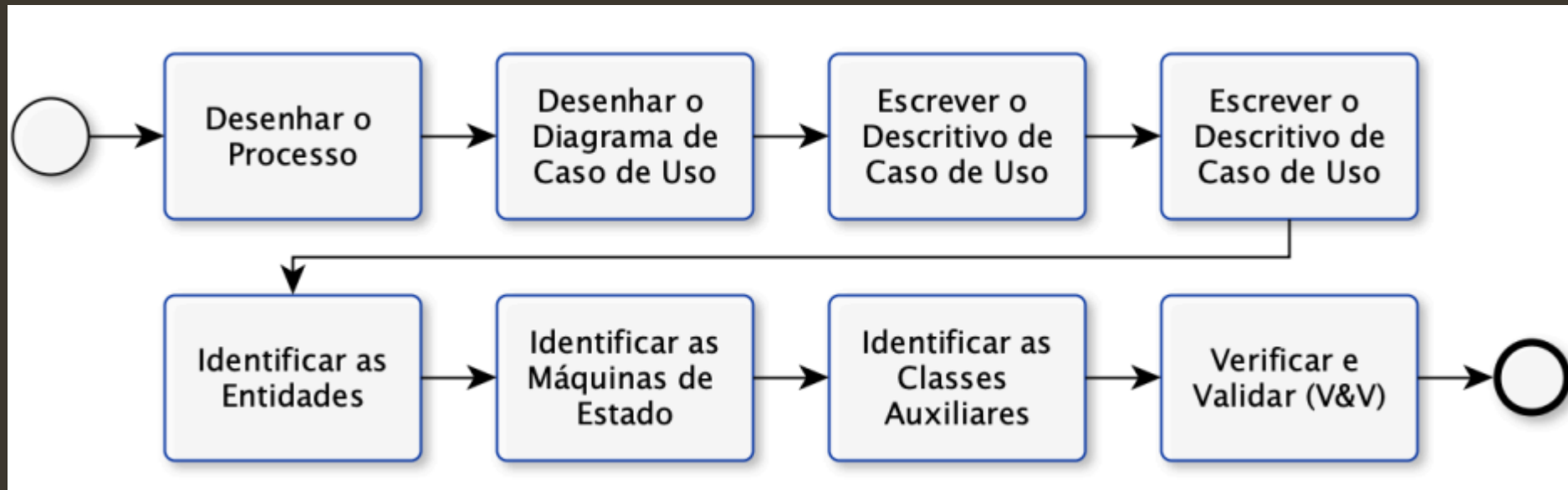
Nesse estudo observamos algumas propostas que tem objetivo de ajudar no direcionamento quanto ao uso prático de análise e projeto OO.

Introdução do tópico 2 – Um caminho para análise e projeto

Nesse nosso estudo vamos avaliar um caminho que podemos seguir para colocar a análise e projeto em condições de início de desenvolvimento.

Um caminho para análise e projeto

- Abaixo podemos ver uma imagem que simplifica bem esse caminho. É importante observar que não se trata de uma execução em cascata.
- Exemplo: VV não deve ser ao final, mas sim a cada atividade.



Conclusão do tópico 2 – Um caminho para análise e projeto

Nesse nosso estudo vimos um caminho que pode ser seguido e é importante observar que apesar de, inicialmente, adicionar custo ao projeto seus benefícios pagam integralmente esse investimento.

Introdução do tópico 3 – **Boas práticas de desenho**

Nesse estudo iremos avaliar uma forma de aplicação das práticas o que estudamos até aqui.

Boas práticas de desenho

- Ao projetar software precisamos saber como criar as classes e principalmente como as mesmas se relacionam entre si.
- Para isso os padrões de projeto e análise podem ajudar a minimizar impactos em alterações futura.
- Os padrões de projetos são muito conhecidos, mas os de análise nem tanto;
- Existem padrões de análise em praticamente qualquer domínio. Martin Fowler escreveu um excelente livro que cobre padrões classificados como segue (FOWLER, 1996):
 - Padrões de *Accountability* (organização e responsabilidade)
 - Padrões de Observações e Medições
 - Padrões de Observações para a Finança Corporativa
 - Padrões de Inventário e Contabilidade
 - Padrões de Planejamento
 - Padrões para o Comércio
 - Padrões de Contratos de Derivativos

Conclusão do tópico 3 – Boas práticas de desenho

- Nesse estudo percebemos que o uso de padrões de projeto e análise podem nos ajudar muito na criação de soluções escalares;

Conclusão da Unidade 5

- Nesse estudo abordamos os cenários mais comuns de análise e projetos orientados a objeto.
- Estudamos também como utilizar as técnicas de maneira prática via um processo que possui a essência de vários outros como o RUP. Apresentamos também alguns dos padrões de análise que geralmente não são tão conhecidos como os padrões de projeto.
- Com isso, esperamos aumentar sua caixa de ferramenta sobre o assunto e que isso possa melhorar a forma como você desenvolve software.
- Como uma última recomendação, utilize os padrões com sabedoria e sempre para melhorar a qualidade do software e não para adicionar mais complexidade sem a necessidade.

Conclusão

- Análise e projeto OO
 - Fundamentos de OO e agilidade;
 - Critérios de análise;
 - Critérios de projetos;
 - Alguns tópicos que podem enriquecer o processo, como BPMN, CMMN...;
 - Cenário de utilização.

Obrigado!

- Prof. Me. Ricardo Francisco de Pierre Satin, ITIL, CSM, PMP
- @ricardosatin
- <https://www.linkedin.com/in/ricardosatin/>
- <http://ricardosatin.blogspot.com.br/>
- rfpsatin@yahoo.com.br
- www.ricardosatin.com.br